LAB: GPIO 7-segment

Date: 2026-09-23

Author/Partner: Haeun Yang

Demo Video:

Introduction

In this lab, you are required to create a simple program to control a 7-segment display to show a decimal number (0~9) that increases by pressing a push-button.

Requirement

Hardware

- MCU
  - NUCLEO-F411RE
- Actuator/Sensor/Others:
  - eval board

Software

- PlatformIO, CMSIS, EC_HAL library

Exercise

Port/Pin	Description	Register setting
Port B Pin 5	Clear Pin5 mode	GPIOB->MODER &=~(3<<(5*2))
Port B Pin 5	Set Pin5 mode = Output	GPIOB->MODER |=1<<(5*2)
Port B Pin 6	Clear Pin6 mode	GPIOB->MODER &=~(3<<(6*2))
Port B Pin 6	Set Pin6 mode = Output	GPIOB->MODER |=1<<(6*2)
Port B Pin Y	Clear PinY mode	GPIOB->MODER &=~(3<<(Y*2))
Port B Pin Y	Set PinY mode = Output	GPIOB->MODER |=1<<(Y*2)
Port B Pin 5~9	Clear Pin5~9 mode	GPIOB->MODER &=~(3<<(5*2)) GPIOB->MODER &=~(3<<(62)) GPIOB->MODER &=~(3<<(72)) GPIOB->MODER &=~(3<<(82)) GPIOB->MODER &=~(3<<(9*2))
	Set Pin5~9 mode = Output	GPIOB->MODER |=1<<(52) GPIOB->MODER |=1<<(62) GPIOB->MODER |=1<<(72) GPIOB->MODER |=1<<(82) GPIOB->MODER |=1<<(92)
Port X Pin Y	Clear Pin Y mode	GPIOX->MODER &=~(3<<(Y*2))
	Set Pin Y mode = Output	GPIOX->MODER |=1<<(Y*2)
Port B Pin5	Set Pin5 otype=push-pull	GPIOB->OTYPER &= ~(1UL << 5) ;
Port B PinY	Set PinY otype=push-pull	GPIOB-> OTYPER =~(1UL << Y) ;
Port B Pin5	Set Pin5 ospeed=Fast	GPIOB->OSPEEDR &= ~(3UL << (52)) , GPIOB->OSPEEDR |= (2UL << (52))
Port B PinY	Set PinY ospeed=Fast	GPIOB->OSPEEDR &= ~(3UL << (Y2), GPIOB->OSPEEDR |= (2UL << (Y2))
Port B Pin 5	Set Pin5 PUPD=no pullup/down	GPIOB->PUPDR &= ~(3UL << (5*2)) ;
Port B Pin Y	Set PinY PUPD=no pullup/down	GPIOB->PUPDR &= ~(3UL << (Y*2)) ;

Problem 1. Library for 7-segment display on JKIT Board

Problem

There are four 7-Segment Displays on JKIT board.

- You need to select which one to use.
- There is no BCD decoder
- JKIT - Nucleo 64: link

Procedure

1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library **ecGPIO2.h, ecGPIO2.c** are in `EC\include\`.
3. Create a new project under the directory `EC\lab\`

- Environment: "**LAB_GPIO_7segment"**
- Source File: ""**LAB_GPIO_7segment.c"**

Configuration

Example Code

main

```
#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"

PinName_t pinFND[8]    = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7} ;
PinName_t selectFND[4] = {PA_10, PA_11, PC_4, PC_3} ;


void selectFNDsetup(PinName_t *selectFND){
    for (int i=0; i<4; i++){
        GPIO_init(selectFND[i], OUTPUT) ;
        GPIO_otype(selectFND[i], PUSH_PULL) ;
    }
}

void pinFNDsetup(PinName_t *pinFND){
    for (int i=0; i<8; i++){
        GPIO_init(pinFND[i], OUTPUT) ;
        GPIO_ospeed(pinFND[i], MEDIUM_SPEED) ;
        GPIO_otype(pinFND[i], PUSH_PULL) ;
    }
}

void setup(void){
    // Intialize System Clock
    RCC_HSI_init() ;
    // Intialize FND pins and Others
```

```
    FND_init(pinFND);
    FND_select_init(selectFND) ;
    // Intialize  Others
    selectFNDsetup(selectFND) ;
    pinFNDsetup(pinFND) ;
};

int main(void) {
    setup();
    uint8_t numDisplay1  = 0 ;
    uint8_t numDisplay2  = 5 ;
    uint8_t selectFND1  = 0 ;
    uint8_t selectFND2  = 1 ;

    while (1) {

        FND_select(selectFND1);
        FND_display(pinFND, numDisplay1);
        delay_ms_HSI(1);
        FND_select(selectFND2);
        FND_display(pinFND, numDisplay2);
        delay_ms_HSI(1);
    }
}
```

GPIO.h

```
// FND for JKIT board
extern int numberFND[12][8];
void FND_init(PinName_t *pinFND);
void FND_select_init(PinName_t *selectFND);
void FND_display(uint8_t num);
void FND_select(uint8_t digit);
```

GPIO.c

```
// FND - JKIT board
static PinName_t _fndPins[8];
static PinName_t _fndSel[4];

//Each led that has to light up gets a 1, every other led gets a 0
//its in order  { (a,b,c,d,e,f,g,dp) }
int numberFND[12][8]={
                    {1,1,1,1,1,1,0,0},          // zero
                    {0,1,1,0,0,0,0,0},          // one
                    {1,1,0,1,1,0,1,0},          // two
                    {1,1,1,1,0,0,1,0},          // three
                    {0,1,1,0,0,1,1,0},          // four
                    {1,0,1,1,0,1,1,0},          // five
                    {1,0,1,1,1,1,1,0},          // six
                    {1,1,1,0,0,1,0,0},          // seven
                    {1,1,1,1,1,1,1,0},          // eight
                    {1,1,1,1,0,1,1,0},          // nine
                    {0,0,0,0,0,0,0,1},          // dot
```

```
                    {0,0,0,0,0,0,0,0}          // blank
                };


// Initialize DOUT pins for 7 segment leds
void FND_init(PinName_t *pinFND){
    for (int i=0; i<8; i++){
        _fndPins[i] = pinFND[i] ;
        GPIO_init(_fndPins[i], OUTPUT) ;
    }
}

// Initialize select pins (FND0~FND3) as DOUT
void FND_select_init(PinName_t *selectFND){
    for (int i=0; i<FND_NUM; i++){
        _fndSel[i] = selectFND[i] ;
        GPIO_init(_fndSel[i], OUTPUT) ;
    }
}

// Display a number 0 - 9 only
void FND_display(PinName_t *pinFND, uint8_t num){
    if (num > 9 || num < 0) printf("!!ERROR!!") ;
    else {
        for (int j=0; j<8; j++){
            _fndPins[j] = numberFND[num][j] ;
            GPIO_write(pinFND[j], _fndPins[j]) ;
        }
    }
}


// Select display: FND0 to FND3
void FND_select(uint8_t digit){
    for (int i=0; i<FND_NUM; i++){
        if (i == digit) {
            GPIO_mode(_fndSel[i], OUTPUT) ;
            GPIO_write(_fndSel[i], HIGH) ;
        }
        else GPIO_mode(_fndSel[i], INPUT) ;
    }
}
```

FND_init()

```
void FND_init(PinName_t *pinFND)
```

7-세그먼트 LED의 세그먼트(a~g, dp) 핀들을 디지털 출력(DOUT)으로 초기화.

Parameters

- `pinFND` : 8개의 세그먼트 핀 이름을 담은 배열 (예: `{PB_0, PB_1, ..., PB_7}`)

Example code

```
PinName_t pinFND[8] = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};

FND_init(pinFND);
```

FND_select_init()

```
void FND_select_init(PinName_t *selectFND)
```

4개의 자릿수 선택(FND0~FND3) 핀들을 디지털 출력(DOUT)으로 초기화 한다.

Parameters

- `selectFND` : 4개의 자릿수 선택 핀 이름을 담은 배열 (예: `{PA_10, PA_11, PC_4, PC_3}`)

Example code

```
PinName_t selectFND[4] = {PA_10, PA_11, PC_4, PC_3};
FND_select_init(selectFND);
```

FND_display()

```
void FND_display(PinName_t *pinFND, uint8_t num)
```

0~9 사이의 숫자 하나를 세그먼트 패턴으로 변환하여 화면에 표시한다.

Parameters

- `pinFND` : 8개의 세그먼트 핀 이름을 담은 배열
- `num` : 표시할 숫자 (0~9만 유효)

Example code

```
FND_display(pinFND, 7);   // 숫자 '7' 모양을 세그먼트에 표시
```

FND_select()

```
void FND_select(uint8_t digit)
```

4개의 자릿수(FND0~FND3) 중 하나를 선택해서 활성화.

Parameters

- `digit`: 활성화할 자릿수 인덱스 (0~3)

Example code

```
FND_select(2);   // 3번째 자릿수(FND2)를 활성화
```

Problem 2: Counter with Button Press

Procedure

Create a code that increases the displayed number from 0 to 9 with each button press.

- After the number '9', it should start from '0' again.
- Modify from Problem 1 source code

Configuration

Example Code

```
#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"
#include <stdio.h>

#define BUTTON_PIN PA_4

PinName_t pinFND[8]    = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7} ;
PinName_t selectFND[4] = {PA_10, PA_11, PC_4, PC_3} ;

// Initialiization
void selectFNDsetup(PinName_t *selectFND){
    for (int i=0; i<4; i++){
        GPIO_init(selectFND[i], OUTPUT) ;
        GPIO_otype(selectFND[i], PUSH_PULL) ;
    }
}

void pinFNDsetup(PinName_t *pinFND){
    for (int i=0; i<8; i++){
        GPIO_init(pinFND[i], OUTPUT) ;
        GPIO_ospeed(pinFND[i], MEDIUM_SPEED) ;
        GPIO_otype(pinFND[i], PUSH_PULL) ;
    }
}

void setup(void){
    RCC_HSI_init() ;                // Intialize System Clock

    GPIO_init(BUTTON_PIN, INPUT) ;  // Initialize the pushbutton pin as an
input:
    GPIO_pupd(BUTTON_PIN, PU) ;
```

```
    FND_init(pinFND);               // Intialize FND pins and Others
    FND_select_init(selectFND) ;

    selectFNDsetup(selectFND) ;     // Intialize  Others
    pinFNDsetup(pinFND) ;
};

int main(void) {
    setup();
    int numDisplay[10]  = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} ;
    uint8_t selectFND1  = 0 ;
    uint8_t selectFND2  = 1 ;
    int buttonState = 1 ;
    int prevButtonState = 1 ;
    int count = 0 ;
    int countTens = 0 ;
    int whileCounts = 0 ;

    while (1) {
        // count 버튼 누를 때마다 숫자 증가
        // BUTTON -> Active Low(0 일 때가 눌렸을 때)
        buttonState = GPIO_read(BUTTON_PIN) ;
        if (buttonState == 0 && prevButtonState == 1 && count <= 9) {
            FND_select(selectFND1);
            FND_display(pinFND, numDisplay[count]);
            count++ ;
            delay_ms_HSI(1);
        }
        else if (buttonState == 0 && prevButtonState == 1 && count == 10){
            while(1){
                buttonState = GPIO_read(BUTTON_PIN) ;
                if (buttonState == 0 && prevButtonState == 1) countTens++ ;
                if (countTens == 11) {
                    countTens = 0 ;
                    count++ ;
                    break ;
                }
                FND_select(selectFND1);
                FND_display(pinFND, numDisplay[countTens-1]);
                delay_ms_HSI(1);
                FND_select(selectFND2);
                FND_display(pinFND, numDisplay[1]);
                delay_ms_HSI(1);
                prevButtonState = buttonState;
            }
        }

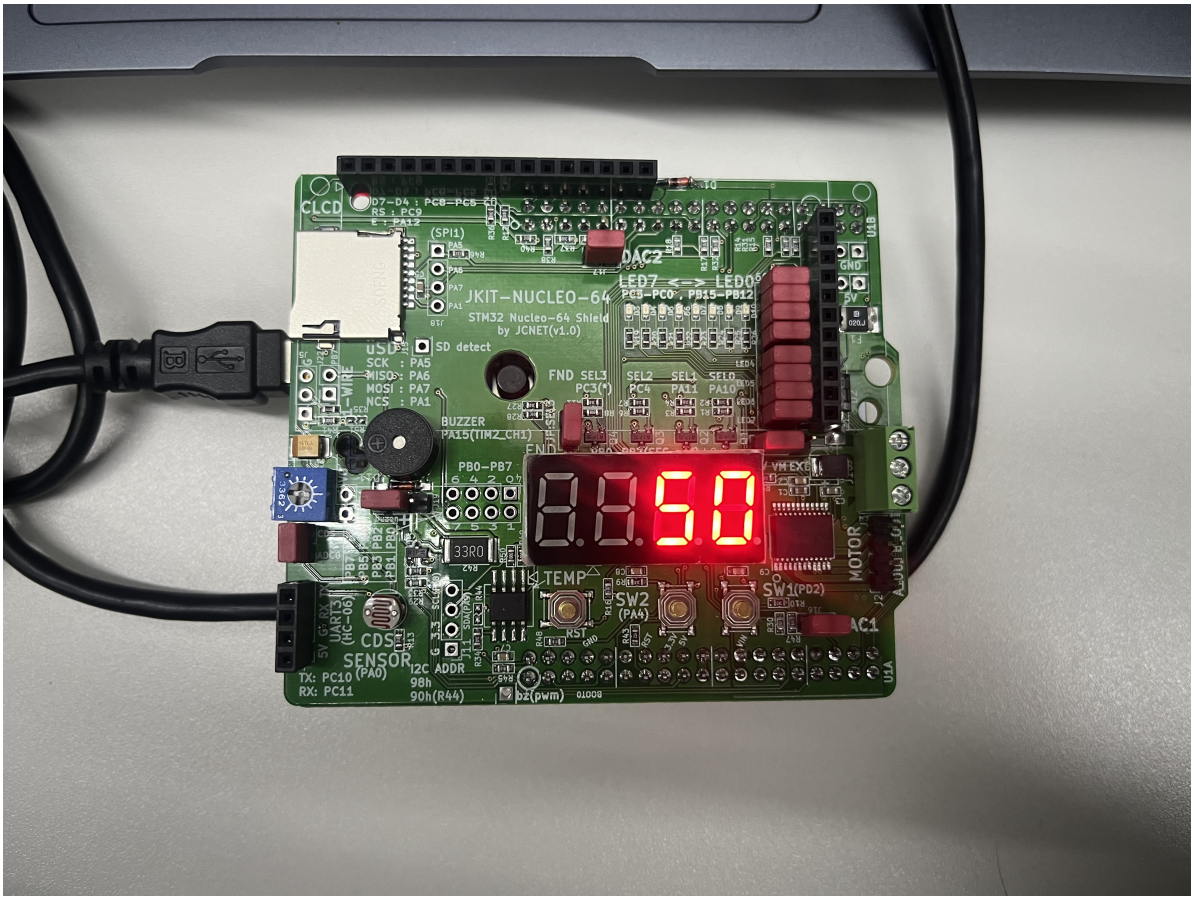
        else if (buttonState == 0 && prevButtonState == 1 && count == 11){
            while(1){
                buttonState = GPIO_read(BUTTON_PIN) ;
                if (buttonState == 0 && prevButtonState == 1) countTens++ ;
                if (countTens == 11) {
                    countTens = 0 ;
                    count++ ;
                    break ;
                }
                FND_select(selectFND1);
```

```
                FND_display(pinFND, numDisplay[countTens-1]);
                delay_ms_HSI(1);
                FND_select(selectFND2);
                FND_display(pinFND, numDisplay[2]);
                delay_ms_HSI(1);
                prevButtonState = buttonState;
            }
        }

        prevButtonState = buttonState ;
        delay_ms_HSI(100) ;
    }
}
```

Results

LAB_GPIO_7segment_P1



####

Discussion

1. Analyze the result and explain any other necessary discussion.

> problem1인 50을 화면에 띄우는 것은 잘 진행되었다. problem2도 display를 하나만 써서 버튼이 눌림에 따라서 숫자를 증가 시켜 화면에 띄우는 것도 순조롭게 진행되었다. 하지만 추가적으로 1에서 19까지는 좀 어려웠다. 두 자리를 띄우고 숫자를 수정하면서 띄우는 것이 어려웠다. 12부터는 고스팅 현상이 나타난다.

1. What are the common cathode and common anode of 7-segment display?

> 7-segment display는 숫자 모양을 만드릭 위해 배치된 8개의 LED로 구성된다. 이 LED들은 내부적으로 두 가지 방식 중 하나로 배선된다.
>
> 첫번째는 common cathode이다. 이는 8개의 LED의 음극 단자가 모두 하나의 공통 핀으로 묶여 있고, 이 공통 핀은 그라운드에 연결된다. 따라서 각 세그먼크를 키기 위해서는 개별 anode 핀에 HIGH 전압을 인가해야 한다.
>
> 두번째는 common anode이다. 이는 8개 LED의 양극 단자가 모두 하나의 공통 핀으로 묶여 있고, 이 공통 핀은 VCC에 연결된다. 각 세그먼트를 켜려면 그라운도로 연결해야 한다.

1. Does the LED of a 7-segment display (common anode) pin turn ON when 'HIGH' is given to the LED pin from the MCU?

> 아니다. common anode 7-segment display에서는 segment 핀에 LOW를 줄 때 LED가 켜지고, HIGH를 주면 꺼진 상태로 유지된다.
>
> 그 이유는 내부 배선 구조에서 알 수 있는데, 모든 anode가 이미 VCC에 묶여 있기 때문이다. 특정 segment의 LED에 전류가 흐르려면 그 LED의 cathode 핀 쪽이 MCU에 의해 더 낮은 전위로 끌어내려져 LED 양단에 전위차가 생겨야 한다.

1. How can we display 4 digit-numbers on the 4 FNDS at the same time? How to remove ghosting of displaying a number?

> 4개의 자릿수가 8개의 segment 선을 전부 공유하고 있기 때문에, 하나의 공유된 segment 버스만으로 4개의 서로 다른 자릿값을 정확히 같은 순간에 표시하는 것은 물리적으로 불가능하다. 하지만 해결책은 time-division multiplexing이다.
>
> 이 방법의 과정을 설명하면, 1번 자리의 선택 선을 활성화 하고, 1번 자리의 세그먼트 패턴을 출력한 뒤, 아주 짧은 시간 유지한다. 그리고 1번 자리를 비활성화하고, 2번 자리의 선택 선을 활성화 한 뒤, 2번 자리의 세그먼트 패턴을 출력하고 짧게 유지한다. 3번과 4번도 같은 방식으로 반복하고 다시 1번 자리로 돌아가 이 과정을 반복한다.
>
> 고스팅은 원래 꺼져 있어야 할 자릿수에 희미하고 의도치 않은 빛이 나타나는 현상이다. 이는 전환 타이밍 문제 때문에 발생한다. 이는, 이전 자릿수의 선택 선이 완전히 비활성화되기 전에 다음 자릿수용 세그먼트 데이터가 공유 버스에 먼저 쓰이거나, 혹은 다음 자릿수의 세그먼트 데이터가 준비되기 전에 다음 자릿수의 선택 선이 먼저 활성화 되는 경이다. 이를 제거하는 일반적인 방법은 자릿수 전환 사이에 화면을 잠깐 공백 처리한다. 다음 자릿수로 전환하고 그에 맞는 세그먼트 패턴을 쓰기 전에, 모든 세그먼트 출력을 잠깐 꺼두더나 비활성화 하는 것이다. 또 다른 방법으로는 선택과 비선택 데이터 갱신의 순서를 올바르게 하는 것이다.

Reference

Complete list of all references used (github, blog, paper, etc)

https://docs.sunfounder.com/projects/uno-mega-kit/en/latest/components/component_7_segment.html

https://www.electronics-tutorials.ws/counter/7-segment-display.html

https://www.nutsvolts.com/magazine/article/using-seven-segment-displays-part-2

https://www.egr.msu.edu/classes/ece480/capstone/fall10/group02/doc/Application_Note_-_Nick.pdf